

EE351

微机原理与微系统

姜俊敏

电子与电子工程系

第二章 微型计算机基础知识

- 本章学习目标:

- 1. 掌握微型计算机中的数制及其编码
- 2. 掌握布尔代数和常见逻辑电路
- 3. 了解微型计算机的常用技术术语和技术

第二章 微型计算机基础知识

●2.1 微型机中的数制及其编码

- 2.1.1 数与数制
- 2.1.2 不同数制之间的转换
- 2.1.3 数制数据的编码及其运算
 - 1.原码; 2.反码; 3.补码; 4.十进制数的编码((非)压缩BCD码)
- 2.1.4 非数值数据的编码 (ASCII码, 汉字编码)

●2.2 布尔代数和常见逻辑电路

●2.3 微型计算机的常用技术术语和技术

- 2.3.1 常用单位及术语;
- 2.3.2 常见技术

2.1 微型机中的数制及其编码

2.1.1 数与数制

进位计数制，简称数制。

十进制：人们习惯采用的计数制是十进制。

0—9 十个不同的基数，逢十进一。

用D(Decimal)表示或省略。

二进制：计算机所采用的计数制是二进制。

只有0、1两个不同的基数，逢二进一。

用B(Binary)表示。

因为计算机用晶体管截止、饱和两个态下的输出电平1、0表示数字。

2.1.1 数与数制

十进制与二进制转换

人机交互时采用十进制，计算机内数据存储、计算、处理用二进制，需要进行转换。在计算机中的解决方法是，利用接口技术作转换。

- ◇ 如：用键盘输入数据时使用十进制数，即输入电路使用的键盘是十进制数，输入接口电路将十进制数转换为二进制数后送到机器内部。
- ◇ 显示器显示十进制，输入显示器的数据为二进制/十六进制。

2.1.2 不同数制之间的转换

1、十进制数转换为二进制数

十进制数转换为二进制数的方法

- 整数部分转换方法: "除2取余, 逆序排列"

整数(及商)除以2取余, 直到商为0为止。最后将所有余数倒序排列, 得到转换结果。

- 小数部分转换方法: "乘2取整, 顺序排列"

乘以2取整, 直到满足精度要求为止。

例1：将十进制数100转换为二进制数

		余数	
2	100	0	低位
2	50	0	
2	25	1	
2	12	0	
2	6	0	
2	3	1	
2	1	1	高位
	0		

整数部分转换方法：
除2取余，逆序排列

$$(100)_{10} = (01100100)_2$$

或者表示为：
100D=01100100B

例2：将十进制数45.613转换成二进制数

	余数		取整数	高位
$2 \overline{) 45}$	1	低位	$0.613 \times 2 = 1.226$	1
$2 \overline{) 22}$	0		$0.226 \times 2 = 0.452$	0
$2 \overline{) 11}$	1		$0.452 \times 2 = 0.904$	0
$2 \overline{) 5}$	1		$0.904 \times 2 = 1.808$	1
$2 \overline{) 2}$	0		$0.808 \times 2 = 1.616$	1
$2 \overline{) 1}$	1	高位	$0.616 \times 2 = 1.232$	1
0				低位

整数部分转换示意图 小数部分转换示意图

45.613 $\approx$ (101101.100111)₂ 或 45.613D $\approx$ 101101.100111B
(45.613)₁₀

整数部分转换：除2取余，逆序排列
小数部分转换：乘2取整，顺序排列

2、二进制数转换为十进制数

转换方法：按权展开相加。

例如，一个8位二进制数11111111B的各位的权值依次是 2^7 、 2^6 、 2^5 、...、 2^0 。

如将1011 0110B转换为十进制数的方法是：

$$1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \\ = 182 \quad , \text{即}, (10110110)_2 = (182)_{10}$$

或表示为：10110110B=182D=182

3、十六进制数

优点：二进制数位数较多时，读写不方便。而使用十六进制表示简明。

十六进制数表示：

0~9、A、B、C、D、E、F十六个不同的基数
逢十六进一，用H（Hexadecimal）表示。

◇ 二进制数转换为十六进制数

原理：四位二进制数对应一位十六进制，所以二进制整数转换为16进制时，从最低位开始，每四位一组（不足四位时高位补0）转换成一位十六进制数据即可。

例如：1011 0110B=B6H； 0111 1100B=7CH。

十六进制数转换为二进制数

十六进制数据转换为二进制数据时，把每一位十六进制数据直接写成四位二进制数。例如：64H=0110 0100B

表2-1 4位二进制数和一位十六进制数具有一一对应关系

十六进制	二进制	十六进制	二进制
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

十六进制数据和十进制数据之间的转换

- 通过二进制进行相互转换；

- 十进制数转换成十六进制：

将十进制整数除以十六取余，小数部分乘以十六取整，可直接转换为十六进制数；

- 十六进制数转换成十进制：

将十六进制数按权展开相加得到十进制数。 $n+1$ 位十六进制数的权值分别为 16^n 、 16^{n-1} 、...、 16^2 、 16^1 、 16^0

2.1.3 数制数据的编码及其运算

在计算机中，数据分无符号数和带符号数。

无符号数用整个机器字长的全部二进制位表示数值位，无符号位；

带符号数用最高位表示该数的符号位。

$$\text{8位: } (100)_{10} = (01100100)_2$$

带符号数又有原码、补码和反码三种形式。

$$\text{8位字长: } (-100)_{10} = (11100100)_2$$

$$\text{8位字长: } (+100)_{10} = (01100100)_2$$

►本小节介绍：1. 原码；2. 反码；3. 补码；4. 十进制数的编码（(非)压缩BCD码）

1、原码表示法

由于计算机中只能有0、1两种数，不仅数的数值部分在计算机中用0、1编码的形式表示，**正、负号也只能用0、1编码表示。**

一般用数的**最高位**(Most Significant Bit, MSB)**表示数的正负符号。**

$$\text{8位字长: } (-100)_{10} = (\textcolor{red}{1}1100100)_2$$

$$\text{8位字长: } (+100)_{10} = (\textcolor{red}{0}1100100)_2$$

例如，若用**5位二进制数**表示数据时，**最高位表示符号**，0表示正数，1表示负数，余下的**四位表示数据**：

MSB=0表示正数，如+**1011B**表示为**01011B**；

MSB=1表示负数，如-**1011B**表示为**11011B**。

机器数和真值

- 把一个数在机器内的二进制形式称为**机器数**。
- 把这个数本身称为该机器数的**真值**。

前面“**01011B**”和“**11011B**”就是两个机器数。

它们的真值分别为**+1011B**和**-1011B**。

- ◇ **真值为纯小数时**，其原码形式为 $X_S.X_1X_2\cdots X_n$ ，其中 X_S 表示符号位。例如，

若 $X=+0.0110B$ ，则 $[X]_{\text{原}}=0.0110B$ ；

若 $X=-0.0110B$ ，则 $[X]_{\text{原}}=1.0110B$

- ◇ **真值为纯整数时**，其原码形式为 $X_SX_nX_{n-1}\cdots X_2X_1$ ，其中 X_S 表示符号位。 例 $[X]_{\text{原}}=00110B$ ； $[X]_{\text{原}}=10110B$

1、原码表示法

8位二进制原码的表示范围为 $-2^7-1 \sim +2^7-1$:

$$-127 \sim -0 \sim +0 \sim +127$$

16位二进制原码的表示范围为 $-2^{15}-1 \sim +2^{15}-1$:

$$-32767 \sim -0 \sim +0 \sim +32767$$

原码表示中, 真值0有两种不同的表示形式(8位字长):

$$[+0]_{\text{原}} = 0000\ 0000\text{B}, \quad [-0]_{\text{原}} = 1000\ 0000\text{B}$$

无符号数

没有符号位的数，称为无符号数。

如字长为8位时，能表示的无符号数的最大值为11111111B，即255(2^8-1)，而8位有符号数的最大值是01111111B，即+127(2^7-1)。

无符号数的最大值比有符号数大一倍。

8位二进制无符号数的表示范围为：0~255 (2^8-1)

16位二进制无符号数的表示范围为：0~65535 ($2^{16}-1$)

原码优缺点

- ◇ 原码的优点是直观易懂，机器数和真值间的转换很容易，用原码实现乘、除运算的规则简单。缺点是加、减运算规则较复杂。

直接用0、1表示正、负，运算时带来的问题：在计算机内有符号数和无符号数的表示形式并没有任何区别，所以，CPU在进行运算时，并不知道参与运算的数是有符号数还是无符号数，

在进行有符号数的运算时，会将符号也当作是数值进行运算，因而有时会出现错误的结果。

原码计算存在的问题

两个正数相加时，符号位也同时相加

例1，两个有符号正数01010111B（87D）和 00010110B（22D）相加。

	01010111
+	00010110
	01101101

其和为01101101B，即十进制的109 < 127，
符号位为0，表示和为正数，结果正确。

- ◇ 若两个数之和不超出其所能表示的最大值127时，符号位相加： $0+0=0$ ，即和仍然为正数，结果正确。

原码计算存在的问题

若两个正数之和超出了其所能表示的最大值127时，就会产生数字位向符号位的进位，两符号位相加 $0+0=0$ ，再加上低位进上来的1，则符号位为1，作为有符号数，表示两个正数相加的和为负数，显然是不对的。

例2，两个有符号正数00110111B（55D）和01011101B（93D）相加。

和应为+148，但符号位为1，表示和是负数，错误。

	00110111
+	01011101
	10010100

产生错误的原因是：相加的和 $148 > 127$ ，超出了8位有符号正数所能表示的最大值，对于有符号数，这种数值运算侵入到符号位造成结果错误的情况，称为溢出。

原码计算存在的问题

一个正数与一个负数相加，和的符号位不应是两个符号位直接运算的值： $0+1=1$ 。

和的符号位应由两数中绝对值大的数所决定。

两个负数相加时，由于 $1+1=10$ ，符号位只表示0，因此和的符号也不应是由两符号位直接运算的结果所决定。

为解决机器内有符号数的符号位参加运算的问题，引入了反码和补码。

2、反码表示法

对正数来说，其反码和原码相同。

即 $[X]_{\text{原}} = [X]_{\text{反}}$ 。

对负数来说,求反码过程: 保持(负数原码的)符号位1不变，数值部分各位取反。

真值、原码、反码的不同表示:		
真值 X	$+1101\text{B}$	-1101B
原码 $[X]_{\text{原}}$	01101B	11101B
反码 $[X]_{\text{反}}$	01101B	10010B

◇ 在反码表示中，真值0也有两种不同的表示形式：

$$[+0]_{\text{原}} = 00000\text{B} \quad [-0]_{\text{原}} = 10000\text{B}$$

$$[+0]_{\text{反}} = 00000\text{B}, \quad [-0]_{\text{反}} = 11111\text{B}$$

2、反码表示法

反码运算要注意以下三个问题：

- 符号位可与数值位一样参加运算。
- 符号位运算后如有进位产生，则把这个进位送回到最低位去相加，这叫循环进位。
- 反码运算具有性质： $[X]_{\text{反}} + [Y]_{\text{反}} = [X + Y]_{\text{反}}$

3、补码表示法

(1)同余的概念

给定一个正整数 M ，如果两整数 A 与 B 满足 $A-B$ 能够被 M 整除，即 $(A-B)/M$ 得到一个整数，那么就称整数 A 与 B 对模 M 同余。 M 称为模，对模 M 同余是整数的一个等价关系：

可写成： $A=B \pmod{M}$ 或 $A \equiv B \pmod{M}$

例如：对钟表来说,其模 $M=12$,故4点和16点、5点和17点...均是同余的。

$$4=16 \pmod{12}, 5=17 \pmod{12}$$

(2) 补码的概念

指针式钟表的校准（本应3点显示5点，快两个小时） **$(\text{mod } 12)$**

- 方法一: 往回拨两个小时。 $5-2=3$
- 方法二: 往前拨10个小时，结果相同。 $5+10(-12)=3$

若大于12点，可舍弃(减)12，12点也计为0点。

◇ 用钟表计时相当于取模为12，数一旦大于或等于其模，模就被自动舍弃，所以， $5+10(-12)=3$ 。

◇ 以12为模时，-2和10同余： $-2=10(\text{mod } 12)$ ，
即 $5-2=5+10(\text{mod } 12)$ 。

◇ 有同余关系的两个数为互补关系，其中一个称为另一个的补码，-2与10对模12来说互补，即10可看作-2的补码， $-2+12(\text{模})=10$ 。

(2) 补码的概念

- ◆ 知道模的大小，求某个负数的补码时，只要将该负数加上其模，就得到它的补码。

- ◆ 如以“10”为模，“-7”的补码为

$$(-7) + 10 = 3 \pmod{10}$$

这时“3”就是“-7”的补码。

◇ 当正数加上一个负数时，实际是做减法。引入模、补码概念之后，可将该正数加上这个负数的补码，这样就可把减法运算用变成加法运算。

例如，当模为10时，

$$7 + (-4) = 7 + (-4 + 10) = 7 + 6 = 13 = 13 - 10 = 3 \pmod{10}$$

数一旦大于或等于其模，模就被自动舍弃

(3) 以 2^n 为模的补码

在计算机中，带符号的数用二进制补码表示。存放数据的存储器的二进制位数都是确定的。如每个存数单元的字长为 n 位，则它的模就是 2^n

如8位单片机, $n=8$,模是 $2^8=256$,即1 0000 0000B (9位)

2^n 是 $n+1$ 位的二进制数100...0B（1后面有 n 个0），由于机器只能表示 n 位数，因此数 2^n 在机器中仅能以 n 个0来表示，而该数最高位的数字1就被自动舍弃了。

即以 2^n 为模时 2^n 和0在机器中的表示形式完全一样。

以 2^n 为模的补码

D7	D6	D5	D4	D3	D2	D1	D0
1	1	1	1	1	1	1	1

➤ 如果将 n 位字长的二进制数的最高位留做符号位:

$X = X_{n-1}X_{n-2}X_{n-3}\dots X_1X_0$, (如 $n=8$ 时, $X_7X_6X_5X_4X_3X_2X_1X_0$)

$2^{n-1}2^{n-2}2^{n-3} \dots 2^12^0$

各位权值 $2^72^62^52^42^32^22^12^0$

则数字只剩下 $n-1$ 位, 下标从 $n-2$ 到 0 。

数字 X 的补码 (注意: 以 2^n 为模) 的表示形式为:

当 x 为正数时, 即 $X = +X_{n-2}X_{n-3}\dots X_1X_0$ 时,

$$[X]_{\text{补}} = 2^n + X$$

$$= 0X_{n-2}X_{n-3}\dots X_1X_0 \pmod{2^n}$$

$$=[X]_{\text{原}} \quad 2^n = 10 \ 0 \ 0 \ \dots \ 0 \ 0$$

$$X_{n-1} = 0$$

符号位

(因数只有 n 位, 最高位数字1被自动舍弃)

补码举例

例如， $n=8$ 时，模为 $2^8=1\ 0000\ 0000B$ ，则

$-101\ 0111B$ (-87)的补码为：

$$\begin{aligned} [-101\ 0111B]_{\text{补}} &= 1\ 0000\ 0000B - 101\ 0111B \\ &= 1010\ 1001B \end{aligned}$$

$$\begin{aligned} \text{或} [-101\ 0111B]_{\text{补}} &= [-101\ 0111B]_{\text{反}} + 1 \\ &= 1\ 010\ 1000B + 1 \\ &= 1\ 010\ 1001B \end{aligned}$$

$1\ 0000\ 0000B$
$- \quad 101\ 0111B$
$\hline 1010\ 1001B$

补码的真值计算方法： $-2^7 + 2^5 + 2^3 + 1 = -128 + 32 + 8 + 1 = -87$

3、补码表示法

➤正数: 补码和原码的形式相同:

$$[X]_{\text{原}} = [X]_{\text{补}};$$

➤负数: 补码为其反码(原码的数值部分各位求反,而保持符号位1不变)加1。

例如:	X	$[X]_{\text{原}}$	$[X]_{\text{反}}$	$[X]_{\text{补}}$
正数	+000 1101B	0000 1101B	0000 1101B	0000 1101B
负数	-000 1101B	1000 1101B	1111 0010B	1111 0011B

原码数值位取反

求补

负数求反码补码过程: **1111 0010B** **1111 0010B +1**

反码与补码的性质

➤ 不论是正数，还是负数，反码与补码具有下列性质：

$$\begin{aligned} [[X]_{\text{反}}]_{\text{反}} &= [X]_{\text{原}} \\ [[X]_{\text{补}}]_{\text{补}} &= [X]_{\text{原}} \end{aligned}$$

可用来方便地求
反码和补码的真值

证明：对正数： $[X]_{\text{原}} = [X]_{\text{反}} = [X]_{\text{补}} = X$ (数值不变)，

$$[[X]_{\text{反}}]_{\text{反}} = [X]_{\text{反}} = [X]_{\text{原}}$$

同理： $[[X]_{\text{补}}]_{\text{补}} = [X]_{\text{补}} = [X]_{\text{原}}$

所以上式总成立。

对负数：(负数求反过程：保持符号位1不变，数值位求反。)

易证 $[[X]_{\text{反}}]_{\text{反}} = [X]_{\text{原}}$

(负数求补过程：先求反(符号位1不变，数值位求反)，
再加1。)

反码与补码的性质

➤ $[[X]_{\text{补}}]_{\text{补}} = [X]_{\text{原}}$ 对负数的证明:

证明: 对负数: $[X]_{\text{原}} = \underbrace{2^{n-1}}_{\text{符号位}} + \underbrace{X_{n-2}X_{n-3}\cdots X_1X_0}_{\text{数值位}}$

$$[X]_{\text{补}} = 2^n - X_{n-2}X_{n-3}\cdots X_1X_0$$

$$= 2^{n-1} + (2^{n-1} - X_{n-2}X_{n-3}\cdots X_1X_0)$$

符号位

数值位求反+1

$$\begin{aligned} [[X]_{\text{补}}]_{\text{补}} &= 2^{n-1} + \underbrace{2^{n-1} - (2^{n-1} - X_{n-2}X_{n-3}\cdots X_1X_0)}_{\text{数值位求反+1}} \\ &= 2^{n-1} + X_{n-2}X_{n-3}\cdots X_1X_0 = [X]_{\text{原}} \end{aligned}$$

(负数求补就是保持符号位1不变, 数值位求反+1。)

反码与补码的性质

➤ 不论正数,还是负数,反码与补码具有下列相似的性质:

$$\begin{aligned} [[X]_{\text{反}}]_{\text{反}} &= [X]_{\text{原}} \\ [[X]_{\text{补}}]_{\text{补}} &= [X]_{\text{原}} \end{aligned}$$

可用来方便地求
反码和补码的真值

【例2-1】求+13和-13的原码、反码、补码以及反码的反码和补码的补码如下:

X	$[X]_{\text{原}}$	$[X]_{\text{反}}$	$[X]_{\text{补}}$	$[[X]_{\text{反}}]_{\text{反}}$	$[[X]_{\text{补}}]_{\text{补}}$
+0001101b	00001101b	00001101b	00001101b	00001101b	00001101b
-0001101b	10001101b	11110010b	11110011b	10001101b	10001101b
		求反↓ 10001101b	求补↓ 10001100b+1		

表2-2 8位二进制数的原码、反码、补码的表示

无符号数		有符号数			
十进制数	二进制数	真值	原码	反码	补码
127	0111 1111B	+127	0111 1111B	0111 1111B	0111 1111 B
...	...	:	:	:	:
1	0000 0001B	+1	0000 0001B	0000 0001B	0000 0001 B
0	0000 0000B	+0	0000 0000B	0000 0000 B	0000 0000 B
128	1000 0000B	-0	1000 0000B	1111 1111 B	0000 0000 B
129	1000 0001B	-1	1000 0001B	1111 1110 B	1111 1111 B
...	...	:	:	:	:
255	1111 1111B	-127	1111 1111B	1000 0000 B	1000 0001 B
		-128	不能表示	不能表示	1000 0000 B

可见，8位字长原码、反码表示的数的范围：+127~−127，而补码表示的数的范围：+127~−128。

Quiz

- 1、8位二进制原码的表示范围是（ ）
- 2、8位二进制反码的表示范围是（ ）
- 3、8位二进制补码的表示范围是（ ）
- 4、-0的8位二进制补码为（ ）

Quiz

- 1、8位二进制原码的表示范围是 ($-127 \sim +127$)
- 2、8位二进制反码的表示范围是 ($-127 \sim +127$)
- 3、8位二进制补码的表示范围是 ($-128 \sim +127$)
- 4、-0的8位二进制补码为 (0000 0000B)

(4) 数值数据的运算

采用补码进行加减运算时要注意以下几个问题：

➤ ① 溢出

补码运算时,其符号位与数值部分一样参加运算,但结果不能超出其所能表示的数的范围,否则会出现溢出错误。

$$\begin{array}{rcl} \text{例:} & [X]_{\text{补}} = & 00101101 \quad (45D) \\ & + [Y]_{\text{补}} = & 11111111 \quad (-1D) \\ \hline & [X]_{\text{补}} + [Y]_{\text{补}} = & 100101100 \quad (44D) \end{array}$$

- ◇ (4.6.1加减运算指令中) 指出如何判断没有溢出：
符号位(最高位)和次高位上同时存在进位或无进位。

(4) 数值数据的运算

➤ ①溢出

无符号数的加减运算结果超出数的范围的情况叫做**进位或借位**。计算机中有专用的标志位作记录，只要适当处理这些标志，结果就不会出错，所以在多字节数的加减运算时必须考虑进位和借位的处理。

程序状态标志寄存器(PSW)各位的定义

位号	D7	D6	D5	D4	D3	D2	D1	D0
符号	CY	AC	F0	RS1	RS0	OV	F1	P

CY: 进位标志

OV: 溢出标志

(4) 数值数据的运算

◇ 采用补码进行加减运算时要注意的几个问题:

➤②采用了补码以后，符号运算后如出现进位，则把这个进位舍去不要，不影响运算结果，运算后的符号就是结果的符号（在没有溢出的条件下）。

➤③补码运算的性质:

$$[X]_{\text{补}} + [Y]_{\text{补}} = [X + Y]_{\text{补}}$$

$$[X]_{\text{补}} - [Y]_{\text{补}} = [X - Y]_{\text{补}}$$

➤以上运算性质，与数的位数n无关。

例:

$$\begin{array}{r} [X]_{\text{补}} = 00101101 \quad (45D) \\ + [Y]_{\text{补}} = 11111111 \quad (-1D) \\ \hline [X]_{\text{补}} + [Y]_{\text{补}} = 100101100 \quad (44D) \end{array}$$

数值数据的运算 举例

结果不能超出其所能表示的数的范围,否则会出现溢出错误

以二进制数补码运算为例,说明有符号数的运算性质

【例2-2】已知: $X = +0101101B(45D)$, $Y = -0000001B$,
求 $X + Y = ?$ (即两正数相减 $45-1$)

$$[X]_{\text{补}} + [Y]_{\text{补}} = [X+Y]_{\text{补}}$$

$$[X+Y]_{\text{原}} = [[X+Y]_{\text{补}}]_{\text{补}} = [[X]_{\text{补}} + [Y]_{\text{补}}]_{\text{补}} = 00101100B$$

解:

$$\begin{array}{r} [X]_{\text{补}} = 00101101 \quad (45D) \\ + [Y]_{\text{补}} = 11111111 \quad (-1D) \\ \hline [X]_{\text{补}} + [Y]_{\text{补}} = 100101100 \quad (44D) \end{array}$$

进位舍去不要

◇ 加法算减法: 利用补码, 可把减一个正数的运算转变为加上该负数的补码的加法运算。

数值数据的运算 举例

【例2-3】已知: $X = -0001101B$,

$Y = -0000001B$, 求: $X + Y = ?$

解: $[X]_{补} = 11110011$

$+ [Y]_{补} = 11111111$

$[X]_{补} + [Y]_{补} = 111110010$

进位舍去不要

$$[X]_{补} + [Y]_{补} = [X+Y]_{补} = [-0001110B]_{补} = 11110010B$$

所以, $[X+Y]_{原} = [-0001110B]_{原} = 10001110B = [[X+Y]_{补}]_{补}$

$$X+Y = -0001110B$$

$[[X+Y]_{补}]_{补} = [X+Y]_{原}$ 总是成立

数值数据的运算 举例

【例2-4】 已知： $X = +1$, $Y = -128$

求： $X + Y = ?$

$$\begin{array}{r} \text{解：} \quad [X]_{\text{补}} = 00000001 \\ + [Y]_{\text{补}} = 10000000 \\ \hline [X + Y]_{\text{补}} = 10000001 \end{array}$$

$$[X]_{\text{补}} + [Y]_{\text{补}} =$$

因 $[[X + Y]_{\text{补}}]_{\text{补}} = [X + Y]_{\text{原}} = 1111\ 1111\text{B} = [-127]_{\text{原}}$

所以, $X + Y = -111\ 1111 = -127$

严格说, 此题 $[[X + Y]_{\text{补}}]_{\text{补}} = X + Y$ 不成立,
而 $[[X+Y]_{\text{补}}]_{\text{补}} = [X+Y]_{\text{原}}$ 总是成立,

加法器就能完成所有的算术运算

加法算减法：因为减去一个正数的减法运算可以看作是加上一个负数的加法运算，所以在计算机中，求得补码之后，就把减一个正数的运算转变为加上该负数的补码的加法运算。

加法算乘法：可以采用移位相加的方法完成。

加法算除法：采用移位相减的方法完成，这样只用加法器就能完成所有的算术运算。

4、十进制数的编码

常用的十进制数编码有：

BCD码(Binary-Coded Decimal), 余3码, 格雷码

BCD码：是二进制编码形式的十进制数。即用**4位二进制数**表示一位**十进制数**，这种编码形式可以有多种，其中最自然、最常用的一种形式为**8421BCD码**。

8.4.2.1 分别是4位二进制数从左向右的位权值：

D3	D2	D1	D0
1	0	0	1

各位的权值： 2^3 2^2 2^1 2^0

(示例表示**9**)用**4位二进制数**表示**1位十进制数字**0-9

非压缩BCD

- 用一个字节的8位二进制数表示一位十进制数时：即每个字节的高四位为0，只用其低四位表示一位十进制数，则称为非压缩的BCD码，表示格式如图所示。它所表示的数的范围是0-9。

D7	D6	D5	D4	D3	D2	D1	D0
0	0	0	0	1	0	0	1

(示例表示9)用低4位表示1位十进制数字0-9

图2-3 非压缩BCD码的表示格式

例：十进制数3可表示为 $(0000\ 0011)_{8421} = 0000\ 0011B$

压缩BCD

若将1个字节8位用于表示两位十进制数，则称为压缩的BCD码，表示格式如图所示。它所表示的数的范围是0-99。

D7	D6	D5	D4	D3	D2	D1	D0
十位				个位			

图2-4 压缩BCD码的表示格式

例：十进制数39可表示为 $(0011\ 1001)_{8421} = 0010\ 0111B$

例如，用BCD码表示十进制数4321

- 用非压缩BCD码表示,需用4个字节:

00000100, 00000011, 00000010, 00000001;

- 写成十六进制的形式: 04H, 03H, 02H, 01H;
- 用压缩的BCD码表示需2个字节: 0100 0011, 0010 0001;
- 写成十六进制的形式: 43H, 21H。

➤ 尽管在8421码中0~9, 10个数码表示形式与用二进制表示形式一样, 但这是两个完全不同概念, 不能混淆。

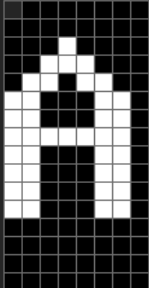
➤ 如, 十进制数39可表示为 $(0011\ 1001)_{8421}$ 或0010 0111B, 两者是完全不同的。

2.1.4 非数值数据的编码

计算机不仅能对数值数据进行处理，还能够对文本和其它非数值数据信息进行处理。

非数值数据是指不能进行算术运算的数据，如：字符、字符串、图形符号和汉字、语音与图像等多种数据。

这些信息在传送时，不是直接传送和处理其原值，而是先按照某种规则进行一定的处理，以便使之具有通用的传送格式。经过这种处理的数值信息，称为编码。

英文字模	位代码	字模信息
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00
	0 0 0 1 0 0 0 0	0x10
	0 0 1 1 1 0 0 0	0x38
	0 1 1 0 1 1 0 0	0x6c
	1 1 0 0 0 1 1 0	0xc6
	1 1 0 0 0 1 1 0	0xc6
	1 1 1 1 1 1 1 0	0xfe
	1 1 0 0 0 1 1 0	0xc6
	1 1 0 0 0 1 1 0	0xc6
	1 1 0 0 0 1 1 0	0xc6
	1 1 0 0 0 1 1 0	0xc6
	1 1 0 0 0 1 1 0	0xc6
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00

(A的ASCII码41h)

A的点阵字模
显示或打印时调用点阵或矢量字库

1、ASCII编码

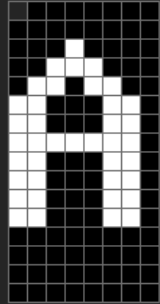
处理文本文件时,每个字符都由其相应的标准字模(例A的字模)构成,文本文件本身并不包括这些字模,而只使用其编码(A的ASCII码41h)来表示每个字符。

例如,使用区位编码的中文编辑时,4位十进制区位码可以表示一万个不同的字符。

国际上通用的标准字符编码为ASCII码 (American Standard Code for Information Interchange, ASCII), 即美国标准信息交换码。

附录A为ASCII码字符表。

(A的ASCII码41h)

英文字模	位代码	字模信息
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00
	0 0 0 1 0 0 0 0	0x10
	0 0 1 1 1 0 0 0	0x38
	0 1 1 0 1 1 0 0	0x6c
	1 1 0 0 0 1 1 0	0xc6
	1 1 0 0 0 1 1 0	0xc6
	1 1 1 1 1 1 1 0	0xfe
	1 1 0 0 0 1 1 0	0xc6
	1 1 0 0 0 1 1 0	0xc6
	1 1 0 0 0 1 1 0	0xc6
	1 1 0 0 0 1 1 0	0xc6
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00
	0 0 0 0 0 0 0 0	0x00

1602液晶模块内部的字符发生存储器 (CGROM)已经存储了160个不同的点阵字符图形,这些字符有:阿拉伯数字、英文字母的大小写、常用的符号、和日文假名等,每一个字符都有一个固定的代码,比如大写的英文字母“A”的代码是01000001B (41H),显示时模块把地址41H中的点阵字符图形显示出来,我们就能看到字母“A”

1、ASCII编码

ASCII表

A的ASCII码41h, 或65

(American Standard Code for Information Interchange 美国标准信息交换代码)

高四位 低四位		ASCII控制字符											ASCII打印字符													
		0000					0001						0010		0011		0100		0101		0110		0111			
		0					1						2		3		4		5		6		7			
		十进制	字符	Ctrl	代码	转义字符	字符解释	十进制	字符	Ctrl	代码	转义字符	字符解释	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	Ctrl
0000	0	0		^@	NUL	\0	空字符	16	►	^P	DLE		数据链路转义	32		48	0	64	@	80	P	96	`	112	p	
0001	1	1	☺	^A	SOH		标题开始	17	◄	^Q	DC1		设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q	
0010	2	2	☹	^B	STX		正文开始	18	↕	^R	DC2		设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r	
0011	3	3	♥	^C	ETX		正文结束	19	!!	^S	DC3		设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s	
0100	4	4	♦	^D	EOT		传输结束	20	¶	^T	DC4		设备控制 4	36	\$	52	4	68	D	84	T	100	d	116	t	
0101	5	5	♣	^E	ENQ		查询	21	§	^U	NAK		否定应答	37	%	53	5	69	E	85	U	101	e	117	u	
0110	6	6	♠	^F	ACK		肯定应答	22	—	^V	SYN		同步空闲	38	&	54	6	70	F	86	V	102	f	118	v	
0111	7	7	●	^G	BEL	\a	响铃	23	↕	^W	ETB		传输块结束	39	'	55	7	71	G	87	W	103	g	119	w	
1000	8	8	◼	^H	BS	\b	退格	24	↑	^X	CAN		取消	40	(	56	8	72	H	88	X	104	h	120	x	
1001	9	9	○	^I	HT	\t	横向制表	25	↓	^Y	EM		介质结束	41	)	57	9	73	I	89	Y	105	i	121	y	
1010	A	10	◼	^J	LF	\n	换行	26	→	^Z	SUB		替代	42	*	58	:	74	J	90	Z	106	j	122	z	
1011	B	11	♂	^K	VT	\v	纵向制表	27	←	^[	ESC	\e	溢出	43	+	59	;	75	K	91	[	107	k	123	{	
1100	C	12	♀	^L	FF	\f	换页	28	└	^\	FS		文件分隔符	44	,	60	<	76	L	92	\	108	l	124		
1101	D	13	♪	^M	CR	\r	回车	29	↔	^]	GS		组分分隔符	45	-	61	=	77	M	93	]	109	m	125	}	
1110	E	14	🎵	^N	SO		移出	30	▲	^^	RS		记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~	
1111	F	15	🎵	^O	SI		移入	31	▼	^-	US		单元分隔符	47	/	63	?	79	O	95	_	111	o	127	␣	^Backspace 代码：DEL

注：表中的ASCII字符可以用“Alt + 小键盘上的数字键”方法输入。

16:01

2013/08/08

附录 A ASCII 码表

1、ASCII

(American

高四位					
	0000				
	0				
低四位	十进制	字符	Ctrl	代码	转义字符
0000	0	0		^@	NUL \0
0001	1	1	☺	^A	SOH
0010	2	2	☹	^B	STX
0011	3	3	♥	^C	ETX
0100	4	4	♦	^D	EOT
0101	5	5	♣	^E	ENQ
0110	6	6	♠	^F	ACK
0111	7	7	●	^G	BEL \a
1000	8	8	◼	^H	BS \b
1001	9	9	○	^I	HT \t
1010	A	10	◐	^J	LF \n
1011	B	11	♂	^K	VT \v
1100	C	12	♀	^L	FF \f
1101	D	13	♪	^M	CR \r
1110	E	14	🎵	^N	SO
1111	F	15	🎼	^O	SI

注：表中的ASCII字符可以用“

ASCII 值	十六进制	控制字符	ASCII 值	十六进制	控制字符	ASCII 值	十六进制	控制字符	ASCII 值	十六进制	控制字符
0	0	NUL	32	20	(space)	64	40	@	96	60	'
1	1	SOH	33	21	!	65	41	A	97	61	a
2	2	STX	34	22	"	66	42	B	98	62	b
3	3	ETX	35	23	#	67	43	C	99	63	c
4	4	EOT	36	24	\$	68	44	D	100	64	d
5	5	END	37	25	%	69	45	E	101	65	e
6	6	ACK	38	26	&	70	46	F	102	66	f
7	7	BEL	39	27	'	71	47	G	103	67	g
8	8	BS	40	28	(	72	48	H	104	68	h
9	9	HT	41	29	)	73	49	I	105	69	i
10	0A	LF	42	2A	*	74	4A	J	106	6A	j
11	0B	VT	43	2B	+	75	4B	K	107	6B	k
12	0C	FF	44	2C	,	76	4C	L	108	6C	l
13	0D	CR	45	2D	-	77	4D	M	109	6D	m
14	0E	SO	46	2E	.	78	4E	N	110	6E	n
15	0F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL

ckspace
码：DEL

1、ASCII编码

ASCII码共定义了256个代码（从0-255）

- 0-31及127(del)(共33个)为控制字符(ASCII control characters)
- 32(空格)-126为可打印字符(ASCII printable characters)
- 从0-127是标准的ASCII编码
- 从128-255是扩展的ASCII编码,不再是国际标准。

◇ 标准ASCII码：用7位二进制编码表示95个字符

26个大写英文字母：41H—5AH

26个小写英文字母：61H—7AH

10个数字码（0~9）：30H—39H

33个特殊字符：[，+，-，@，|，#等

1、ASCII编码

- 附录A为ASCII码字符表，它用8位二进制数表示字符代码。其基本代码占7位，第8位可用作奇偶校验，
- 通过对奇偶校验位设置“1”或“0”状态，保持8位字节中的“1”的个数总是奇数(称为奇校验)或偶数(称为偶校验)，一般用于字符或数字的串行传送时检测传送过程中是否出错。

2、汉字编码

(1) 一汉字输入编码

汉字信息处理系统一般包括编码、输入、编辑、存储、输出和传输。**编码是关键**。不解决这个问题，汉字就不能进入计算机。**汉字输入编码**是用计算机标准键盘上**按键的不同排列组合**来对汉字进行编码。

- ◇ **常用输入编码有数字、字音、字形和音形编码等。**
 - ◆ **数字编码**：如电报码、**区位码**等。无重码，但难记。
 - ◆ **字音编码**：以**汉语拼音作为编码基础**。简单易学，但重码很高，如搜狗拼音、全拼、双拼等。
 - ◆ **字形编码**：如**五笔字型码**、郑码等。有重码少的优点。
 - ◆ **音形编码**：音形编码吸取了音码和形码的优点，使编码规则简化，重码少。常用的有全息码等。

(2) 汉字国标码

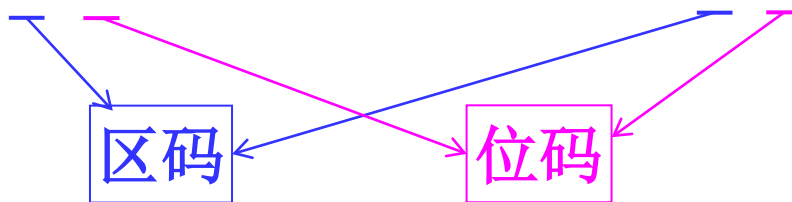
汉字国家标准编码即国标码，是不同汉字信息处理系统间进行汉字交换时所用的编码。以国家标准局GB2312-80规定的汉字交换码作为标准汉字编码。共收录7445个。

汉字区位码：在字符集中,汉字和字符分94个区,每区94位。每个汉字及字符用两个字节表示，前一字节为区码，后一字节为位码，各用两位16进制数字表示。

汉字区位码不等于汉字国标码，两者间关系：

国标码 = 区位码（化成16进制） + 2020H

例："啊"的区位码为"1601"，用16进制表示为1001H，
国标码为3021H。



汉字区位码为何分成94个区, 每区94位? 为何加上 2020H 转化成汉字国标码?

- ◇ 一个字节只用低七位的话有127中状态。英文中0到32(空格)的字符都是些控制字符, 第127位是del字符, 即删除字符, 所以总共有34个控制字符。(从0到127)128减去 34(从0到32是33, 加上第127位的那个字符共34)等于94。所以可供中文使用的是94个状态。
- ◇ 国标码其实就是交换码, 是中国用来交换的, 当然交换码是不能引起歧义的, 94行94列的一个编码再加上32(20H)就是行号从33到126, 列号也是从33到126。
- ◇ 这样就与0~31(20H)的英文控制字符没有冲突了。

(3) 汉字机内码

汉字机内码简称汉字内码,是在计算机外部设备和信息系统内部存储、处理、传输汉字用的代码。

在西文计算机中,无交换码和内码之分,一般以ASCII码作为内码。英文字符的机内码是7位ASCII码,最高位为0 (即D7=0)。

汉字内码用两个字节表示:为区分汉字与英文字符,将汉字国标码每字节的最高位置1,即汉字机内码。

如,“啊”的国标码为0011 0000 0010 0001 (3021H),

机内码为1011 0000 1010 0001 (B0A1H)

汉字机内码=汉字国标码+8080H

(4) 汉字字形码

“你”字的区位码是: 3667;
国标码: 4463H; 机内码: C4E3H

汉字用点阵方式表示其外形, 这个点阵称为**汉字字模**, 也称为**汉字字形码**。

不管汉字笔划多少, 都可在同样的方块中书写, 把方块分割为许多小方块, 组成一个**点阵**, 每个小方块就是点阵中的一点, 即二进制的一位。每个点由**1**和**0**表示“白”和“黑”两种颜色。用这样的点阵就可输出汉字。

16 *16点阵

还有
**24*24点阵、
32*32点阵**

中文字模	位代码	字模信息
	0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0	0x08, 0x80
	0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0	0x08, 0x80
	0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0	0x08, 0x80
	0 0 0 1 0 0 0 0 1 1 1 1 1 1 1 0	0x11, 0xfe
	0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 1	0x11, 0x02
	0 0 1 1 0 0 1 0 0 0 0 0 0 1 0 0	0x32, 0x04
	0 1 0 1 0 1 0 0 0 0 0 1 0 0 0 0	0x54, 0x20
	0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0	0x10, 0x20
	0 0 0 1 0 0 0 0 0 1 0 1 0 1 0 0	0x10, 0xa8
	0 0 0 1 0 0 0 0 0 1 0 1 0 0 1 0	0x10, 0xa4
	0 0 0 1 0 0 0 0 1 0 0 1 0 0 1 0	0x11, 0x26
	0 0 0 1 0 0 1 0 0 0 1 0 0 0 1 0	0x12, 0x22
	0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0	0x10, 0x20
	0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0	0x10, 0x20
	0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0	0x10, 0xa0
	0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0	0x10, 0x40

各种代码间的逻辑关系

不同的输入
编码输入到计
算机中，都统
一使用国标码。

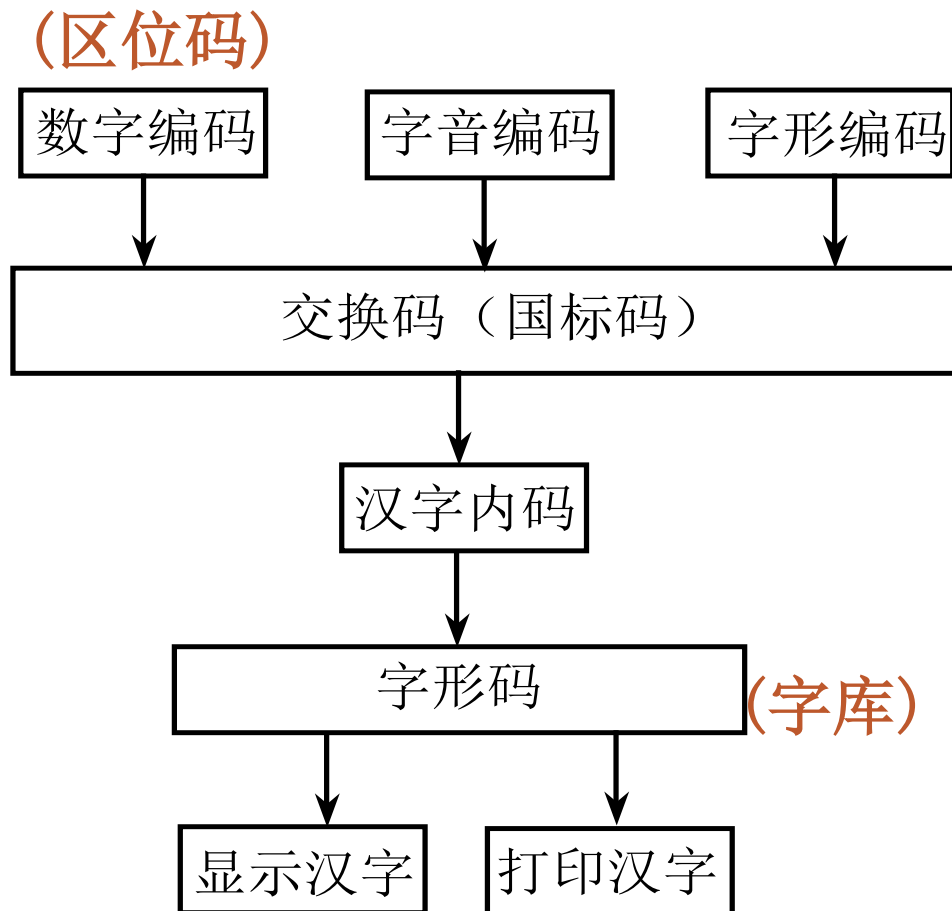


图2-5 各种代码间的逻辑关系

汉字编码 举例

[例2-5] “春”的区位码为20-26, 计算其国标码和机内码。

区位码：	第1字节	第2字节
十进制	20	26
	↓	↓
十六进制	14H	1AH
	+ 20H	+ 20H
国标码：	34H	3AH
	+ 80H	+ 80H
机内码：	B4H	BAH

(5) 汉字编码的发展

汉字编码的发展经历了下面几个阶段：

GB2312-80，国家标准局1980年颁布，标准共收录**6763个汉字**，其中一级汉字3755个，二级汉字3008个；同时，GB 2312收录了包括拉丁字母、希腊字母、日文平假名及片假名字母、俄语西里尔字母在内的**682个全角字符**。共7445个字符。

“通用多八位编码字符集”(Universal Multiple-Octet Coded Character Set)也叫**通用字符集**(Universal Character Set, UCS)，是由**ISO**制定的ISO 10646 (或称ISO/IEC 10646)标准所定义的标准字符集。**1993**年完成，发布了ISO 10646-1:1993，即**Unicode 1.1**，确定了**20902个中日韩CJK统一汉字集**。

国家标准GB18030-2000《信息交换用汉字编码字符集基本集的补充》是我国继GB2312-1980和GB13000-1993之后最重要的汉字编码标准，于2000年3月17日发布，共**27,533个汉字**。

(5) 汉字编码的发展

GB18030-2005 《信息技术中文编码字符集》是我国自主研制的以汉字为主并包含多种我国少数民族文字（如藏、蒙古、傣、彝、朝鲜、维吾尔文等）的超大型中文编码字符集强制性标准，其中收入70,244个汉字。

国家标准GB18030-2005 《信息技术 中文编码字符集》是我国继GB2312-1980和GB13000.1-1993之后最重要的汉字编码标准，是我国计算机系统必须遵循的基础性标准之一。GB18030有两个版本：GB18030-2000和GB18030-2005。GB18030-2000是GBK的取代版本，它的主要特点是在GBK基础上增加了CJK统一汉字扩充A的汉字。GB18030-2005的主要特点是在GB18030-2000基础上增加了CJK统一汉字扩充B的汉字。

(6) 统一代码

统一代码（Unicode）是一种全新的编码方法，此法有足够的能力来表示全世界多达6 800种语言中任意一种语言里使用的所有符号。

其基本方法是，用1个16位的数来表示Unicode中的每个符号，即允许表示65 536个不同的字符或符号。这种符号集被称为基本多语言平面（BMP）。

2.2 布尔代数和常见逻辑电路

2.2.1 布尔代数

布尔代数是英国数学家乔治·布尔（George Boole）发明的，布尔也是数理逻辑的创始人。布尔代数在电子计算机的逻辑设计等工程技术领域中有重要的应用。

◇ 布尔代数以命题为对象，包含三种基本逻辑操作（与，或，非）的完整代数学，它可对命题进行运算。

◇ 基本的运算符有 $+$ ， $\cdot$ 和 $'$ 。

二元运算 $+$ 称为布尔加法，布尔和，布尔并等；

二元运算 $\cdot$ 称为布尔乘法，布尔积，布尔交等；

一元运算 $'$ 称为布尔补，有时使用 “ $\overline{}$ ” 表示。

运算的基本依据是下面的基本公式和规则：

◦ 交换律： $A + B = B + A$

$$A \cdot B = B \cdot A$$

◦ 结合律： $A + (B + C) = (A + B) + C$

$$A \cdot (B \cdot C) = (A \cdot B) \cdot C$$

◦ 分配律： $A + (B \cdot C) = (A + B) \cdot (A + C)$

$$A \cdot (B + C) = A \cdot B + A \cdot C$$

◆ 吸收律： $A + A \cdot B = A$

$$A \cdot (A + B) = A$$

◆ 第二吸收律： $A + \overline{A} \cdot B = A + B$

$$A \cdot (\overline{A} + B) = A \cdot B$$

运算的基本依据是下面的基本公式和规则：

- 反演律(又称摩根定律)：常用于复杂逻辑函数的化简。

$$\overline{A \cdot B \cdot C \cdot \dots} = \overline{A} + \overline{B} + \overline{C} + \dots$$

$$\overline{\overline{A \cdot B \cdot C \cdot \dots}} = \overline{\overline{A}} + \overline{\overline{B}} + \overline{\overline{C}} + \dots$$

◆包含律： $AB + \overline{A}C + BC = AB + \overline{A}C$

$$(A+B)(\overline{A}+C)(B+C) = (A+B)(\overline{A}+C)$$

◆重叠律： $A + A = A, \quad A \cdot A = A$

◆互补律： $A + \overline{A} = 1, \quad A \cdot \overline{A} = 0$

◆0-1律： $0 + A = A, \quad 1 \cdot A = A, \quad 0 \cdot A = 0, \quad 1 + A = 1$

2.2.2 基本逻辑运算和逻辑门电路

- 逻辑运算主要包括与、或、非、异或等。逻辑变量只有两个：逻辑0和逻辑1。
- 逻辑运算只是按位进行运算，没有进位和借位关系，逻辑变量也没有符号问题。

◇ 逻辑与（AND）

逻辑与也叫逻辑乘，其结果叫逻辑积。

运算规则：按位相与，当两逻辑变量都为逻辑1时，逻辑积才为1，否则逻辑积为0。

逻辑与的运算符号：“ $\cdot$ ”或“ $\wedge$ ”。圆点也可省略。

逻辑与（AND）

➤若用Y表示逻辑积，A和B分别表示两个逻辑变量时，其表达式为：

$$Y = A \bullet B$$

A、B及其逻辑积Y的关系的真值表如表2-3所示。

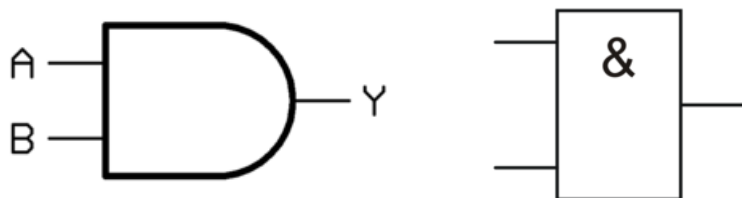
例如

$$\begin{array}{r} 11010110 \\ \wedge \quad 00110101 \\ \hline 00010100 \end{array}$$

表2-3 逻辑与的真值表

A	B	Y=A·B
0	0	0
0	1	0
1	0	0
1	1	1

逻辑与运算对应逻辑电路“与门”。



逻辑或（OR）

逻辑或也叫逻辑加，其结果叫逻辑和。

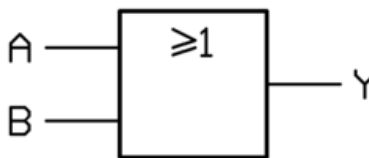
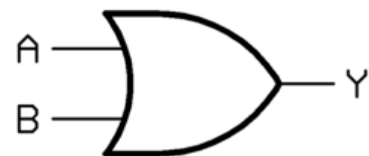
运算的规则：按位运算，两逻辑变量中只要有一个为逻辑1时，逻辑或结果就为1，否则逻辑或结果为0。

逻辑或的运算符号：“+”或者“ $\vee$ ”。

◇若用Y表示逻辑和，A和B分别表示两个逻辑变量时，其表达式为： $Y = A + B$

例

	10010110
$\vee$	00110101
	10110111



“或门”

A	B	$Y=A+B$
0	0	0
0	1	1
1	0	1
1	1	1

表2-4 逻辑或的真值表

逻辑非（NOT）

逻辑非也叫**逻辑反**

运算规则：将一个变量按位求反的运算。

表达式为：

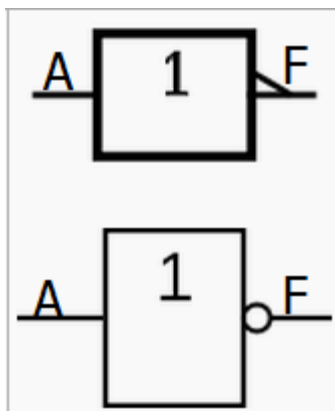
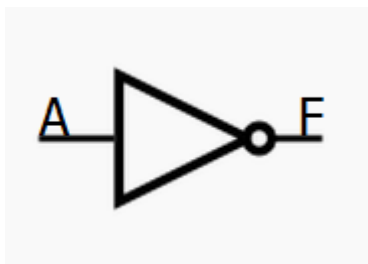
$$Y = \overline{A}$$

表2-5 逻辑非的真值表

A	$Y = \overline{A}$
0	1
1	0

例：若 $A=01100011$ ，则 $Y=\overline{A}=10011100$

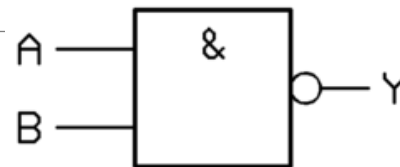
逻辑电路“非门”



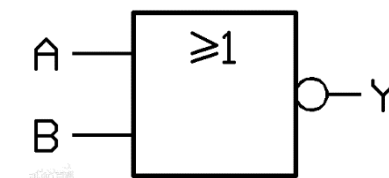
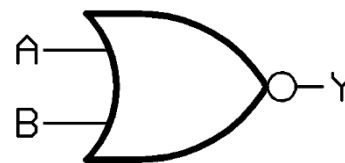
- 上述逻辑运算中，“**非**”运算的级别最高；“**与**”运算次之；“**或**”运算最低。

2、组合逻辑电路

与非门: $Y = \overline{A \bullet B}$



或非门: $Y = \overline{A + B}$



异或门:

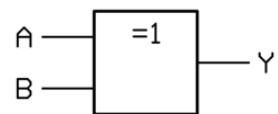
$$Y = A \oplus B = A \bullet \overline{B} + \overline{A} \bullet B$$

异或运算规则：两变量相同为0，不同为1。它可进行两逻辑变量不相等的逻辑测试。

表2-6逻辑异或真值表

例

$$\begin{array}{r} 10110110 \\ \oplus 00110101 \\ \hline 10000011 \end{array}$$



“异或门”

A	B	$Y=A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

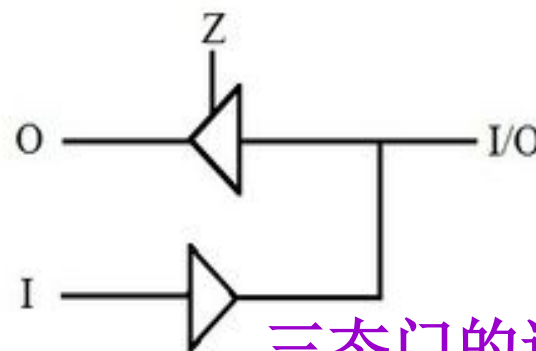
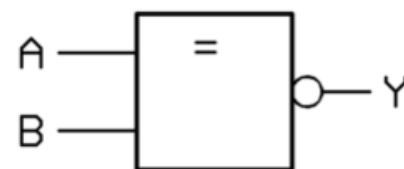
2、组合逻辑电路

“同或门”

同或门 $Y = A \odot B = A \cdot B + \bar{A} \cdot \bar{B}$

逻辑同或真值表

A	B	$Y = A \odot B$
0	0	1
0	1	0
1	0	0
1	1	1



三态门的运用

◇三态门

在数字电路中，有些电路引脚输出状态除了高、低电平即1、0之外，还有第三种状态，即高阻状态，这种状态也叫做浮空、高阻、挂起等。具有高阻态输出的门电路称为三态门，常用于构成总线接收器和发送器。

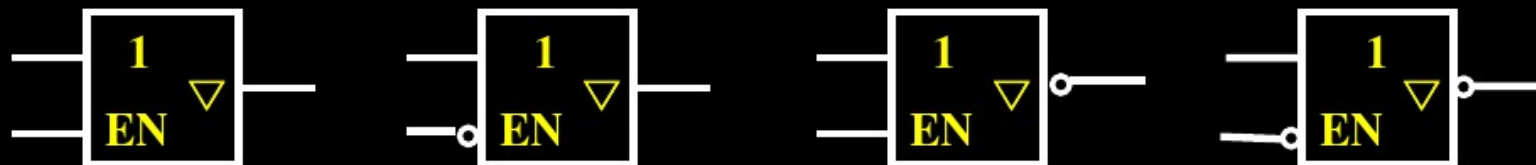
2、组合逻辑电路

三态缓冲器

三态缓冲器（**Three – state Buffer**）又称为三态门、三态驱动器，其三态输出受使能输入端的控制，当使能输入有效时，器件实现正常逻辑状态输出（逻辑0、逻辑1）；当使能输入无效时，输出处于高阻状态，即等效于与所连接的电路断开。

三态缓冲器的逻辑符号

矩形符号



变形符号



高有效使能
原码输出

低有效使能
原码输出

高有效使能
反码输出

低有效使能
反码输出

三态门

高阻状态是门电路输出端的上拉、下拉两个晶体管在控制端的作用下都处于截止状态，而截止状态所呈现的阻抗很高。

当输出端呈高阻状态时，输出端不输出任何逻辑电平，就像是开路一样。

每条线上不能同时有两个信号是输出状态，所以当几个芯片的输出端连在一起的时候，如其中一个是输出状态时，其他芯片的输出端不能同时也是输出状态，此时其他芯片既不能输出1，也不能输出0，就像是断开电路一样，这就是高阻状态，一般用符号Z来表示。

2.2.3 常用组合逻辑电路——1、锁存器

锁存: 把信号暂存以维持某种电平状态。

74LS373: 具有三态缓冲输出的8D锁存器，在单片机应用中，当地址线和数据线复用时，用作地址锁存器。

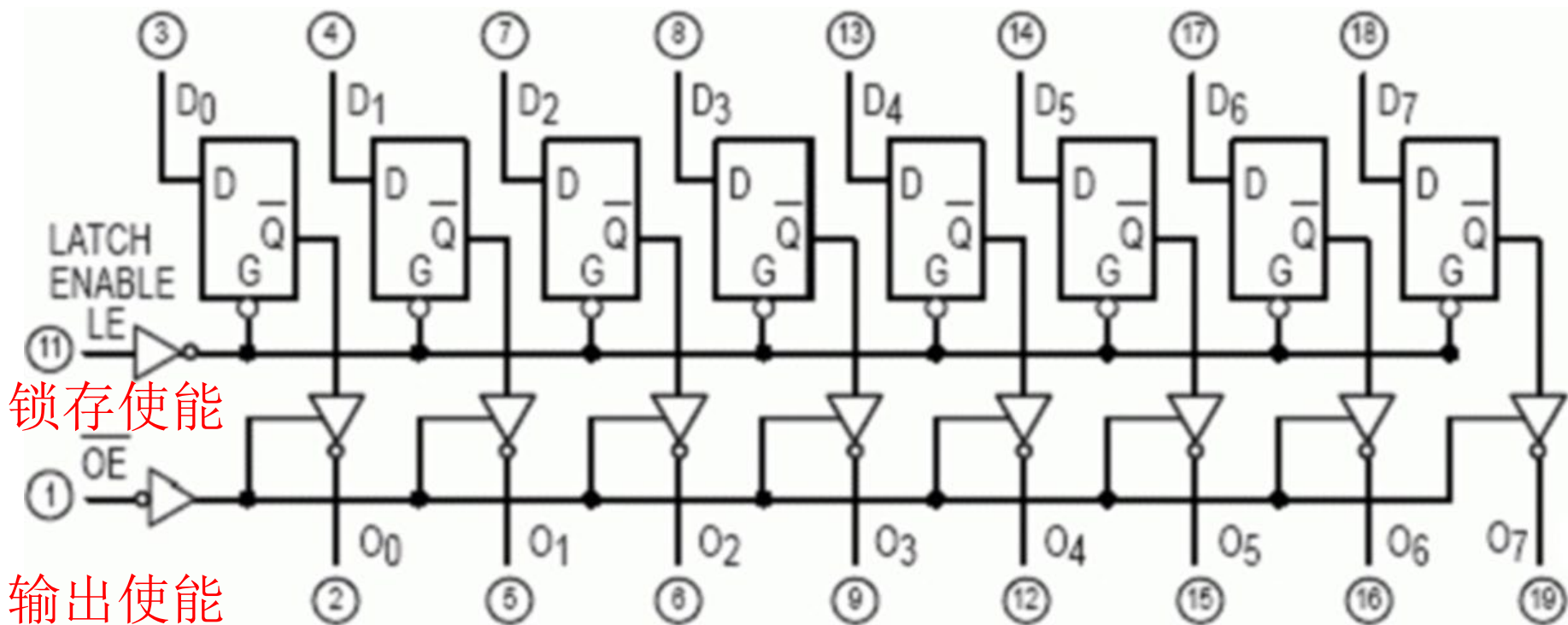


图2-7 74LS373内部逻辑电路图

74LS373引脚图

74LS373引脚图如图2-8所示。

- D0~D7 为数据输入端
- OC为输出允许控制端(三态，低电平有效)；
- G为锁存允许端(也称为LE端)；
- Q0~Q7为输出端。

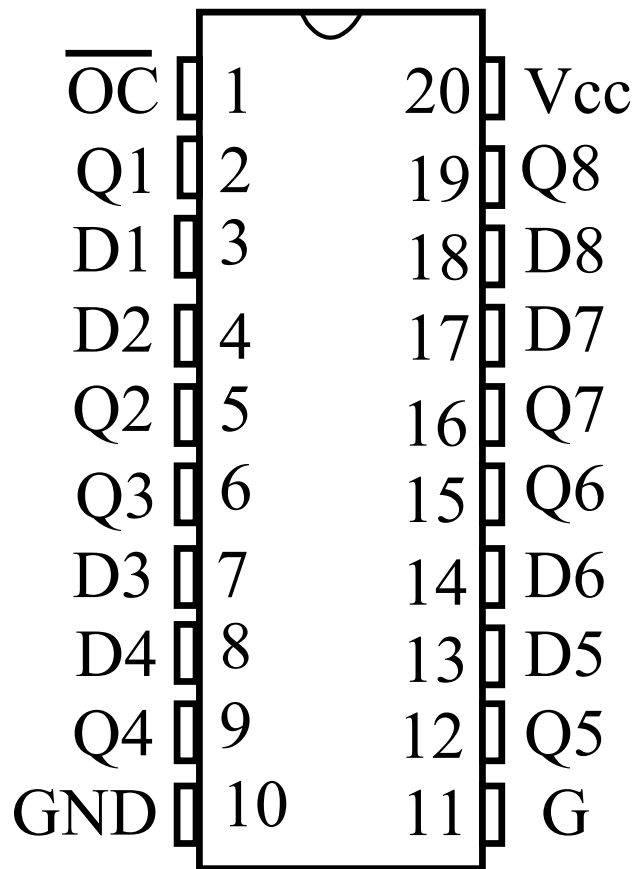


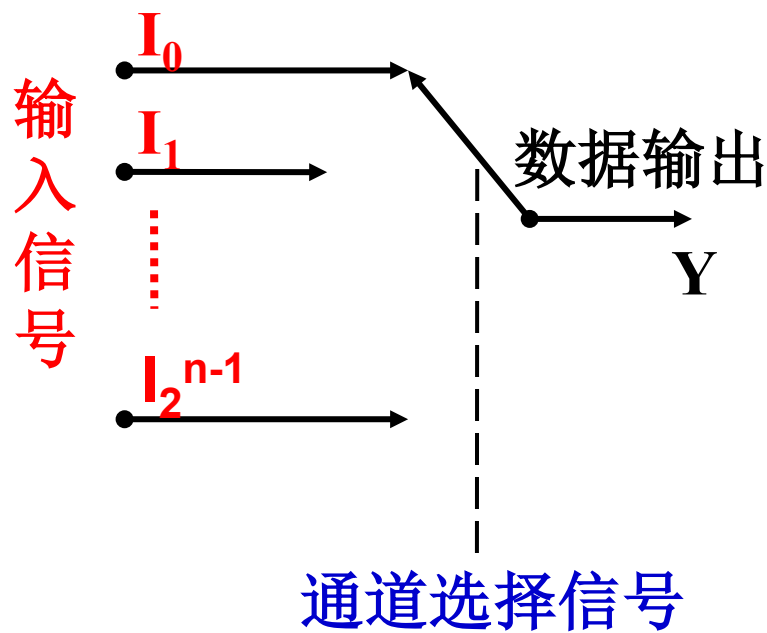
图2-8 74LS373引脚图

2、数据选择器

在数字电路中，常常需要把多个通道的信号传送到公共数据线上，完成这一功能的逻辑电路称为数据选择器。

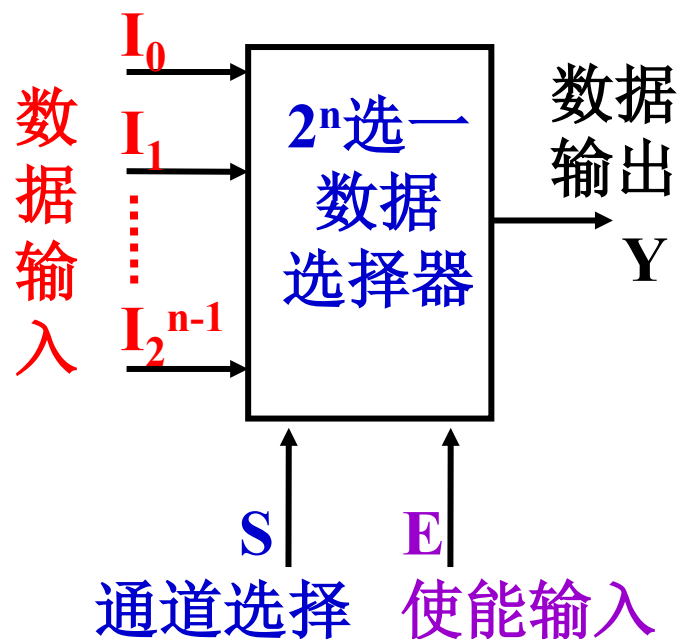
数据选择器又称多路选择器或多路开关，是以“与或”门或“与或非”门为主体的电路，其功能是在通道选择信号的作用下，从多个数据输入通路中选择某一个通道的数据作为该选择器的输出。数据选择器的原理图如图2-9所示。

2、数据选择器



2^n 选一数据选择器

(a)示意图

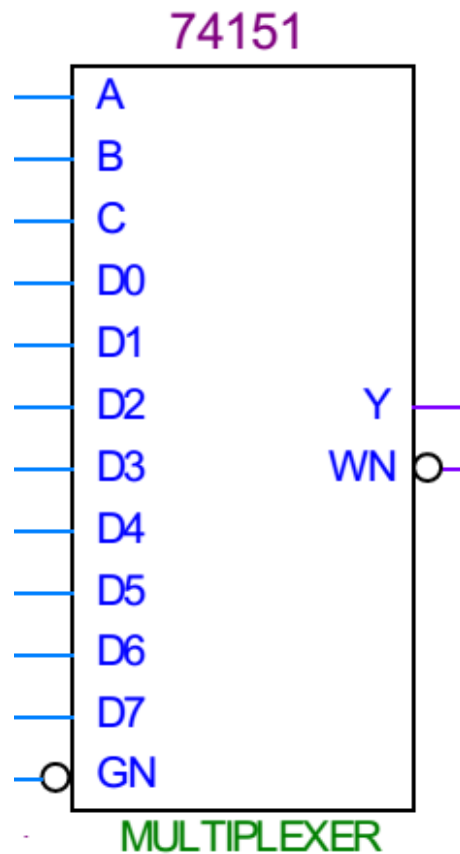


(b)逻辑原理图

图2-9 数据选择器原理图

2、数据选择器

常见的8选1数据选择器有74151、74251；16选1数据选择器有74150，也可以用两片74151连接起来构成。



3、译码器

计算机的数据总线是一组公共信号线，各个芯片分时使用数据总线来传递各自不同的信号。从硬件的角度来看，各个芯片的数据线都是对应引脚**并联**在一起的，那么数据是怎样区分其流向的呢？

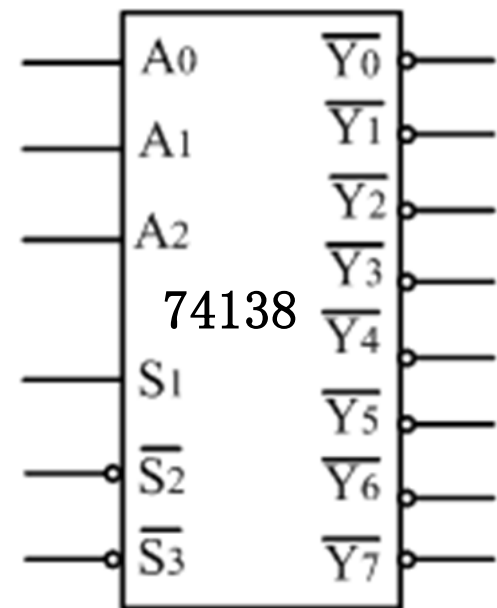
- ◇ 计算机的三总线是数据总线、地址总线和控制总线。
- ◇ 当计算机向一个芯片送出一个数据时，还要通过**地址总线**送出与这个芯片对应**地址码**和相应的**控制信号**，这个地址码是由硬件电路的连接确定的。计算机通过地址总线和**地址译码器**找到确定的芯片，再由控制信号配合，完成对应的数据传送。

译码器电路的功能

把一组输入代码的状态组合翻译成相应输出的控制电位，若输入信号有 n 个，其输出最多可以有 2^n 个。正常输出时， 2^n 个输出中仅有一个输出为低电平(或高电平)，其他 2^n-1 个输出均为高电平(或低电平)。

74138就是常用的译码器电路芯片。

输 入					输 出							
S_1	$\overline{S_2} + \overline{S_3}$	A_2	A_1	A_0	$\overline{Y_0}$	$\overline{Y_1}$	$\overline{Y_2}$	$\overline{Y_3}$	$\overline{Y_4}$	$\overline{Y_5}$	$\overline{Y_6}$	$\overline{Y_7}$
0	X	X	X	X	1	1	1	1	1	1	1	1
X	1	X	X	X	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	1	1	0	1	1	1	1	1	1
1	0	0	1	0	1	1	0	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1	1	1
1	0	1	0	0	1	1	1	1	0	1	1	1
1	0	1	0	1	1	1	1	1	1	0	1	1
1	0	1	1	0	1	1	1	1	1	1	0	1
1	0	1	1	1	1	1	1	1	1	1	1	0



74LS138真值表

译码器电路的功能

74138是具有**3个输入、8个输出**的集成译码器，就是通常所说的三八译码器。3位二进制数字可以有8种不同的组合，或者说，3位二进制地址对应 $2^3=8$ 个不同的地址。

74138译码器的作用就是将输入的三位地址状态转换为8个不同的输出之一，分别控制8个不同电路的芯片。其中**73LS138**芯片是TTL电平的，**74HC138**是CMOS电平的。

片选端/CS信号作用

一个正确的读写操作需要三总线都有信号，缺一不可，且时间上先后顺序配合得当。各个信号在时间上的配合顺序称为**时序**。

一般的接口芯片都有一个**片选端/CS**，有的芯片标为/**CE**，即芯片选择或芯片使能的意思。标示符上面的横线表示低电平有效，就是说该芯片在片选信号为低电平的时候才能进行读写操作。

若片选端无效，即使这个芯片的引脚上有数据信号，控制端也有读写控制信号，对芯片也不能进行数据的读写操作。

使用74138与微控制器的连接如图2-11所示

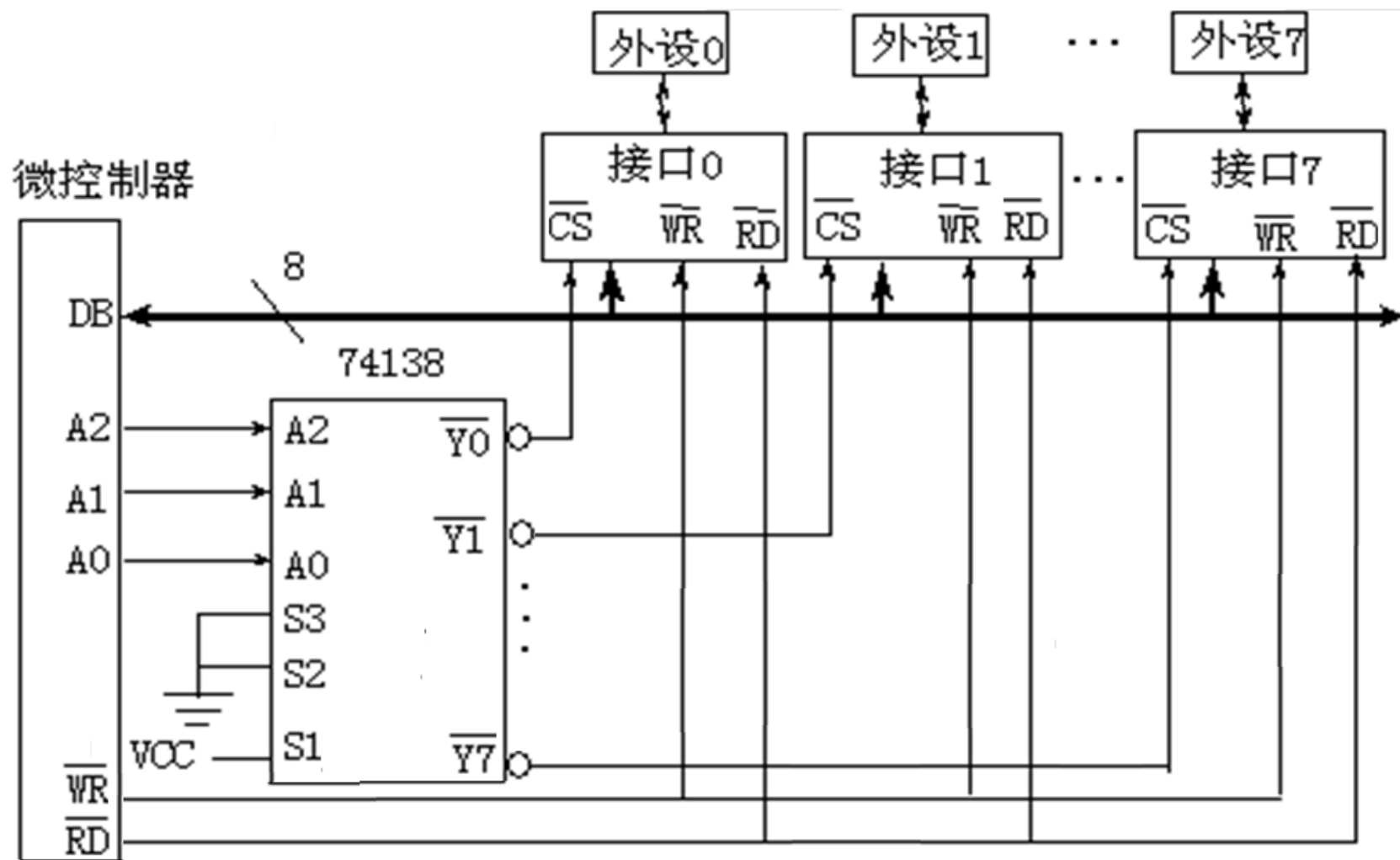


图2-11 译码器与微控制器的连接

74138与微控制器连接图2-11说明

由图2-11可知，数据线、读写控制信号线分别送到8个接口电路的相应管脚上。但是每个接口芯片的片选信号/CS接到了74138不同的输出端。74138的地址输入端接微控制器的地址总线。

根据74138的特性可知,当3位地址为000时, 74138的输出只有 (有效), 均为高电平(无效)。

所以，此时只有接口芯片0被选中，外设0可以进行接下来的读写操作，其他芯片未被选中，所以不会进行读写操作。

$$\overline{Y0} = 0 \qquad \overline{Y1} \sim \overline{Y7}$$

74138与微控制器连接图2-11说明

尽管数据线和控制线都同时接到了每个芯片对应的管脚上，但是片选信号无效的芯片没有对应的操作，只有片选信号端有效的芯片才会有操作。

由此可见，接口0、接口1、.....接口7这8个接口的地址编码分别为000、001、010、011.....111。选择不同的地址，就可以对不同的芯片进行操作，就是各个外设分时使用总线，也就完成了对不同的外设进行读写操作了。

2.3 微型计算机的常用技术术语和技术

2.3.1 常用单位及术语

1、位（bit）

计算机所能表示的**最小**的数字单位，即，二进制数的位。每位只有两种状态**0**、**1**。常用**b**表示。

2、字节（Byte）

8位（bit）为一个字节，是内存的基本单位，常用**B**表示。

2.3.1 常用单位及术语

3、字 (word)

16位二进制数称为一个字，一个字等于两个字节。

4、字长

字长：一般说来，计算机在**同一时间内处理的一组二进制数**称为一个计算机的“字”，而这组二进制数的位数就是“字长”。

字长即字的长度，是**一次可并行处理数据的位数**即**数据线的条数**。常与**CPU内部寄存器, 运算装置, 总线宽度**一致。常用微机字长有8位, 16位和32位。

2.3.1 常用单位及术语

5、数量单位

K 千 **Kilo** 的符号, $1K=1024=2^{10}$

如1KB表示1024个字节

M 兆 **Mega** 的符号, $1M=1K \times 1K=2^{20}$

G 吉 **Giga** 的符号, $1G=1K \times 1M=2^{30}$

T 太 **Tera** 的符号, $1T=1M \times 1M=2^{40}$

[edit]

The BIPM specifies twenty prefixes for the International System of Units (SI).

SI prefixes

V • T • E

Prefix		Base 10	Decimal	English word		Adoption ^[nb 1]	Etymology	
Name	Symbol			Short scale	Long scale		Language	Derived word
yotta	Y	10 ²⁴	1 000 000 000 000 000 000 000 000	septillion	quadrillion	1991	Greek	eight ^[nb 2]
zetta	Z	10 ²¹	1 000 000 000 000 000 000 000	sextillion	trilliard	1991	Latin	seven ^[nb 2]
exa	E	10 ¹⁸	1 000 000 000 000 000 000	quintillion	trillion	1975	Greek	six
peta	P	10 ¹⁵	1 000 000 000 000 000	quadrillion	billiard	1975	Greek	five ^[nb 2]
tera	T	10 ¹²	1 000 000 000 000	trillion	billion	1960	Greek	four ^[nb 2] , monster
giga	G	10 ⁹	1 000 000 000	billion	milliard	1960	Greek	giant
mega	M	10 ⁶	1 000 000	million		1873	Greek	great
kilo	k	10 ³	1 000	thousand		1795	Greek	thousand
hecto	h	10 ²	100	hundred		1795	Greek	hundred
deca	da	10 ¹	10	ten		1795	Greek	ten
		10 ⁰	1	one		–		
deci	d	10 ^{−1}	0.1	tenth		1795	Latin	ten
centi	c	10 ^{−2}	0.01	hundredth		1795	Latin	hundred
milli	m	10 ^{−3}	0.001	thousandth		1795	Latin	thousand
micro	μ	10 ^{−6}	0.000 001	millionth		1873	Greek	small
nano	n	10 ^{−9}	0.000 000 001	billionth	milliardth	1960	Greek	dwarf
pico	p	10 ^{−12}	0.000 000 000 001	trillionth	billionth	1960	Spanish	peak
femto	f	10 ^{−15}	0.000 000 000 000 001	quadrillionth	billiardth	1964	Danish	fifteen, Fermi ^[nb 3]
atto	a	10 ^{−18}	0.000 000 000 000 000 001	quintillionth	trillionth	1964	Danish	eighteen
zepto	z	10 ^{−21}	0.000 000 000 000 000 000 001	sextillionth	trilliardth	1991	Latin	seven ^[nb 2]
yocto	y	10 ^{−24}	0.000 000 000 000 000 000 000 001	septillionth	quadrillionth	1991	Greek	eight ^[nb 2]

1. ^ Prefixes adopted before 1960 already existed before SI. The introduction of the **CGS system** was in 1873.
2. ^ *abcdefp* Part of the beginning of the prefix was modified from the word it was derived from, ex: "peta" (prefix) vs "penta" (derived word).
3. ^ The *fermi* was introduced earlier with the same symbol "fm", in which then the "f" became a prefix. The Danish word is used since it is vaguely spelled similar to *fermi*.

https://en.wikipedia.org/wiki/Metric_prefix

2.3.1 常用单位及术语

6、MIPS (Million Instructions Per Second)

MIPS为单字长定点指令平均执行速度Million Instructions Per Second的缩写，即每秒处理百万条机器语言指令数。这是衡量CPU速度的一个指标。

7、地址

地址是微型计算机存储单元的编号，通常8bit为一个单元，每个单元有独立的编号。存储器地址的最大编号（容量）有限，由地址线的条数决定。如：

◆16条地址线的容量为64KB (0000H~FFFFH); 2^{16}

◆20条地址线的容量1MB (00000H~FFFFFFH); 2^{20}

2.3.1 常用单位及术语

8、总线

CPU是微型计算机的核心。微型计算机利用三总线将CPU与系统的其他部件如存储器、I/O接口等联系起来的。

总线是具有同类性质的一组信号线。

三总线分别是地址总线AB(Address Bus)、数据总线DB(Data Bus)和控制总线CB(Control Bus)。

2.3.1 常用单位及术语

9、访问

CPU对寄存器、存储器或I/O接口电路的**操作**通常分为两类：

把数据存入寄存器、存储器或I/O接口电路的操作称为“**写入**”或**写操作**；

把数据从寄存器、存储器或I/O接口电路取到CPU的操作称为“**读出**”或**读操作**。

这两种**操作过程**通常统称为“**访问**”。

2.3.1 常用单位及术语

8051指令将一个8位二进制数据
30H送到CPU内部的寄存器A中。



10、机器指令

助记符 **MOV A, #30H**

机器指令由二进制代码(例:01110100,00110000)组成,可直接由微处理器进行译码、执行。

一条机器指令应包含要求微处理器所要完成的操作,以及参与该操作的数据或该数据所在的地址,有时还要有操作结果的存放地址信息;

这些都是以二进制数字的形式表示的,当然,也有某些特殊指令不需要数据或地址。

11、汇编指令

机器指令(01110100,00110000)的含义是：将一个8位二进制数据30H送到CPU内部的寄存器A中。

在编程时使用的是比较容易看出其操作含义的、用英文缩写(助记符)形式表示的指令,像上面指令可写成：

MOV A, #30H

- 这种指令叫做汇编指令，这种编程语言称为汇编语言。
- 不同厂家的CPU配有不同格式的汇编语言指令系统，互不兼容，用汇编语言编写的程序叫做汇编语言源程序。

12、指令系统

指令系统是一台计算机所能识别的全部指令的集合。

13、汇编与反汇编

汇编:汇编语言源程序翻译成与之相对应的用二进制数表示的机器语言，才能被微处理器所识别和执行，这种翻译叫做汇编。

MOV A, #30H → 01110100, 00110000

- 汇编语句都可以汇编成对应的机器语言，虽可用人工汇编，但人工汇编麻烦，且易出错，人们编写了**专门的汇编程序来完成汇编**工作，用汇编程序进行汇编就会容易、快速、准确，还能把语法不正确的语句找出来，**利于用户的程序编写和调试**。
- ◇ **反汇编:**将用二进制数表示的机器语言形式的程序翻译成汇编语言形式的源程序的过程叫做反汇编。

14、高级语言

汇编指令虽然较二进制机器指令容易阅读和编写，但还不如高级语言更接近英语自然语言。一般用高级语言编程，再用某种特殊程序翻译成机器语言。

将用高级语言编写的用户程序翻译成某个具体的微处理器的机器语言程序（这种过程叫做编译）的软件，叫做编译器。

现在市面上有各种C编译器，能把C语言转换成某个具体的微处理器的机器语言。

高级语言比较适宜于对汇编语言不熟悉的用户使用，其缺点是不可避免地会出现编译后的机器程序冗长、不够简练，导致程序运行时间长、速度低等问题。

14、高级语言

另外，用汇编语言编程能更有利于硬件电路与程序的结合设计与调试。

当然，如果用户并不在乎程序的长短和运行速度的快慢时，并且拥有对应的编译软件的条件下，采用由C语言编写，经编译程序来进行用户系统的设计和开发，也不失为一种好方法。

2.3.2 常见技术—1、冯·诺依曼结构和哈佛结构

冯·诺依曼结构: 1964年, 冯·诺依曼简化了计算机结构, 提出了“**存储程序**”的思想, 大大提高了计算机的速度。

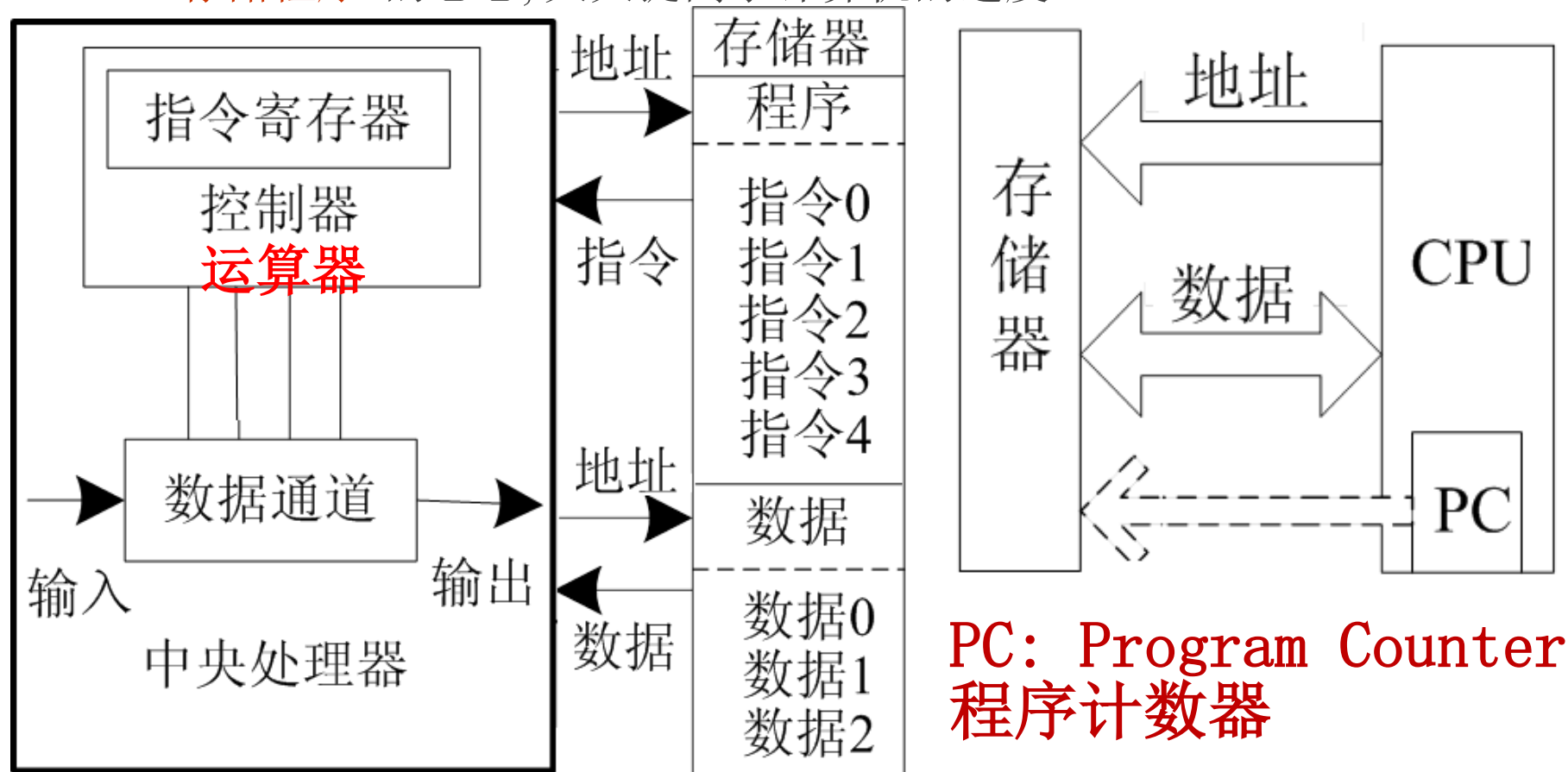


图2-12 冯·诺依曼结构的构成示意图

冯·诺依曼结构

在冯·诺依曼结构中，计算机系统由一个中央处理单元（CPU）和一个存储器组成，数据和指令都存储在存储器中，CPU可以根据所给的地址对存储器进行读或写。程序指令和数据的宽度相同。

Intel 8086、ARM7、MIPS处理器等是冯·诺依曼结构的典型代表。

MIPS的意思是“无内部互锁流水级的微处理器” (Microprocessor without Interlocked Piped Stages)

冯·诺依曼结构

“存储程序”思想可以简化概括为3点：

- 计算机包括运算器、控制器、存储器、输入/输出设备。
- 计算机内部应采用二进制来表示指令和数据。
- 将编写好的程序和数据保存到存储器，然后计算机自动地逐条取出指令和数据进行分析、处理和执行。

哈佛体系结构

各自独立的系统总线

哈佛体系结构中,数据和程序使用各自独立的存储器。程序计数器PC只指向程序存储器而不指向数据存储器,这样做的后果是很难在哈佛体系结构的计算机上编写出一个自修改的程序(有时称为在应用可编程, In Application Programming, IAP)

➤ 哈佛体系结构优点:

- ◆ 独立的程序存储器和数据存储器为数字信号处理提供了较高的性能。
- ◆ 指令和数据可以有不同的数据宽度,具有较高的效率。如TI公司的TMS320 DSP数字信号处理器,飞思卡尔公司的MC68系列、ARM 9、ARM10系列等。

哈佛体系结构

各自独立的系统总线

哈佛体系结构中,数据和程序使用各自独立的存储器。程序计数器PC只指向程序存储器而不指向数据存储器,

➤ 哈佛体系结构优点:

- ◆ 独立的程序存储器和数据存储器为数字信号处理提供了较高的性能。
- ◆ 指令和数据可以有不同的数据宽度,具有较高的效率。如TI公司的TMS320 DSP数字信号处理器,飞思卡尔公司的MC68系列、ARM 9、ARM10系列等。

哈佛体系结构的示意

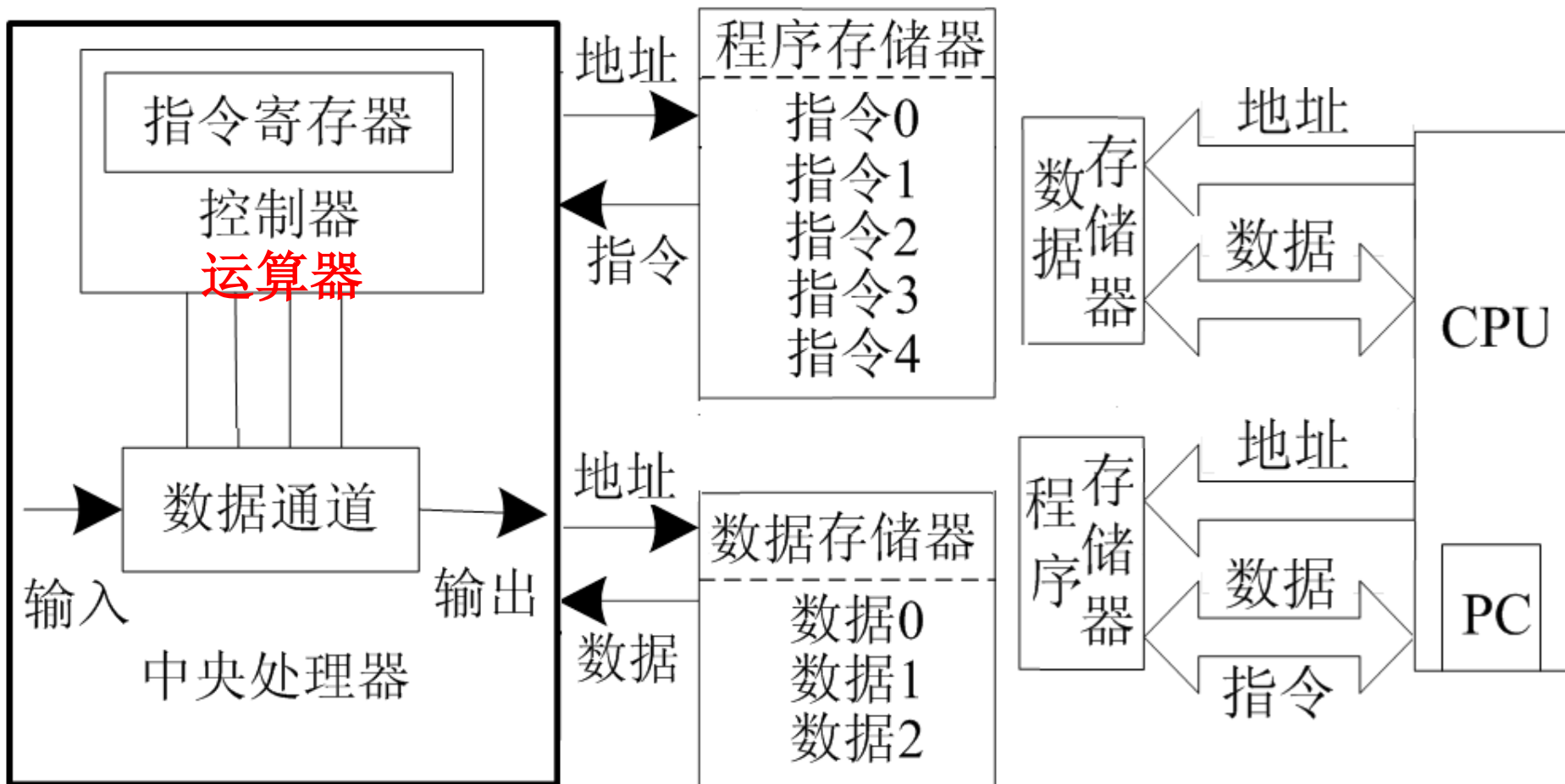


图2-13 哈佛结构的构成示意图

2、高速缓冲存储器Cache

- 为了解决微处理器运行速度快，存储器运行速度慢的矛盾，在两者之间加一级高速缓冲器Cache。
- Cache采用与制作CPU相同的半导体工艺，速度与CPU匹配，其容量约占主存的1%左右。

例如：

- ◇ Intel80386 CPU的高速缓冲存储器在芯片外部，Intel80486 CPU在片内集成了一个8KB的程序和数据Cache，并可以在片外接一个二级高速缓存器。
- ◇ Pentium在芯片内集成了二个8KB的Cache，一个作程序缓存，另一个作数据缓存。

高速缓冲存储器Cache的作用

当CPU要**从主存储器**(个人计算机中称**内存**)中读取一个数据时，**先在Cache中查找是否有该数据**，若有，则从Cache中读取到CPU，**否则从主存储器中读取这个数据送到CPU**，与此**同时**将包含这个数据字的**整个数据块送到Cache中**；

由于存储器访问具有局部性(**程序局部性原理**),在这以后的若干次存储器访问中要读取的**数据就位于刚才取得数据的Cache数据块中的可能性很大**，

只要算法策略得当，Cache的命中率可达**99%**以上，它有效地减少CPU访问低速内存的次数，从而**提高读取数据的速度和整机的性能**。

3、流水线技术

流水线（Pipeline）技术是指在程序执行时多条指令重叠进行操作的一种准并行处理实现技术。

流水线的工作方式：就象工业生产中的装配流水线。在工业制造中采用流水线可以提高单位时间的生产量；同样在CPU中采用流水线设计也有助于提高CPU的效率。

下面以汽车装配为例来解释流水线的工作方式。

3、流水线技术

假设装配一辆汽车需要4个步骤：

- (1)冲压: 制作车身外壳和底盘等部件;
- (2)焊接: 将冲压成形后的各部件焊接成车身骨架;
- (3)涂装: 将车身等部件清洗,处理,打磨,喷漆,烘干;
- (4)总装: 将各部件组装成车。

- ◇ 对应需要专职从事冲压、焊接、涂装和总装的四个人。
- ◇ 采用流水线的制造方式：同一时刻四辆汽车在装配。
- ◇ 如不采用流水线：那么第一辆汽车依次经过上述四个步骤装配完成之后，下一辆汽车才开始进行装配，同一时刻只有一辆汽车在装配。

3、流水线技术

流水线技术可以使用**时空图**来说明。时空图从时间和空间两个方面描述了**流水线的工作过程**。时空图中，横坐标代表时间，纵坐标代表流水线的各个段(级)。

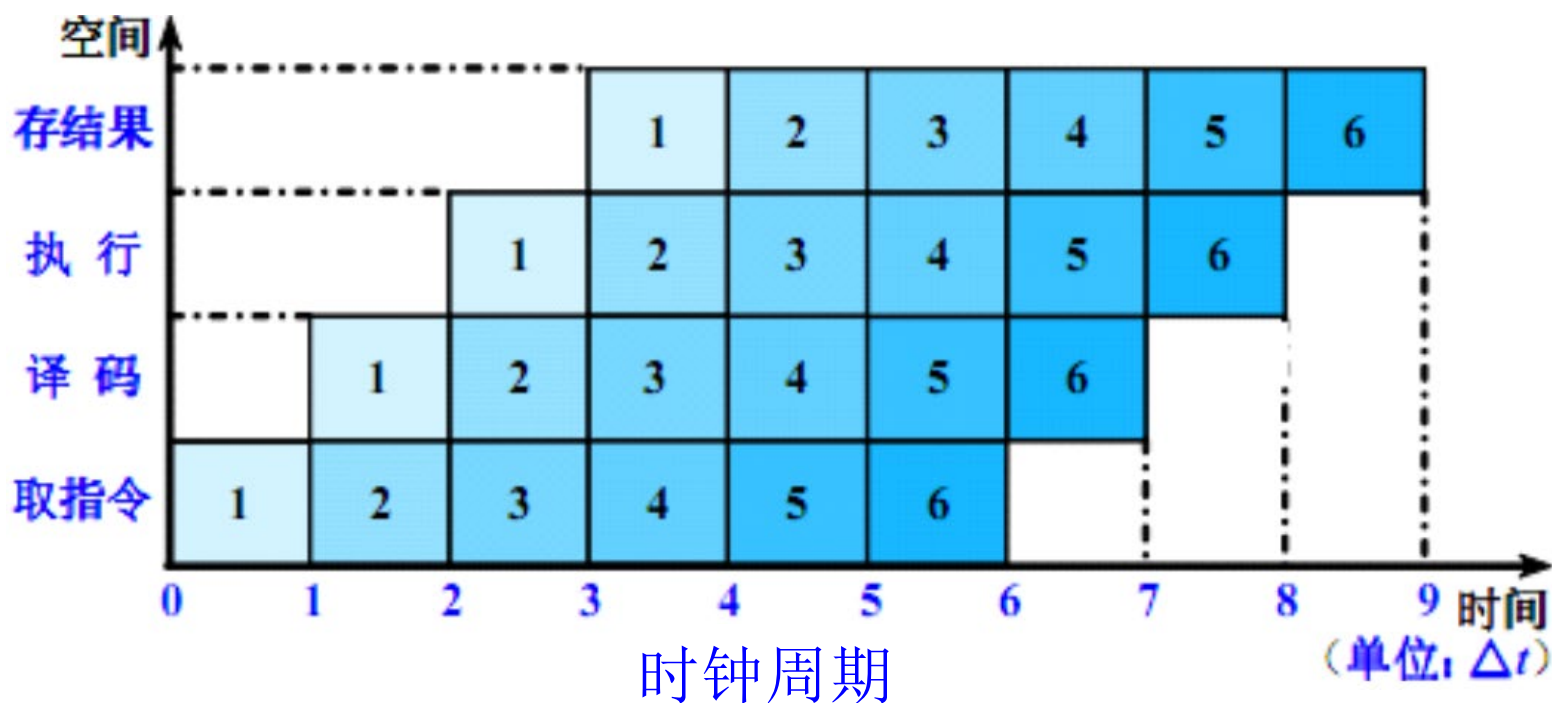


图2-14 4段(级)指令流水线的时空图

3、流水线技术

流水线是Intel首次在Intel 486芯片中开始使用。

在CPU中由5~6个不同功能的电路单元组成一条指令处理流水线，然后将一条X86指令分成5~6步后再由这些电路单元分别执行，这样就能实现在一个CPU时钟周期完成一条指令，因此提高CPU的运算速度。

在Pentium中, 采用两条流水线U和V, 每条都有各自独立的地址生成电路、ALU和连接数据Cache的接口，以便通过各自的数据接口对Cache存取数据。

这样的结构使得Pentium具有更高的执行速度, 达到一个时钟周期执行两条整数运算的指令, 比具有相同时钟频率的前代CPU的实际速度提高一倍。

4、CISC和RISC

CISC:复杂指令集计算机 (Complex Instruction Set Computer),使用大量的指令集,包括复杂指令,可编写短程序,完成复杂功能。

有些复杂指令使用频率较低而造成硬件和资源的浪费。**CISC**结构处理器有一个丰富指令集,每执行一条指令,处理器要在几百条指令中分类查找对应指令,因此需要一定的时间;

- ◇ 由于指令的复杂,增加了处理器的结构复杂性以及逻辑电路的级数,降低了时钟频率,使指令执行的速度变慢,纯CISC结构的处理器执行一条指令至少需要一个以上的时钟周期。

4、CISC和RISC

RISC, Reduced Instruction Set Computer, 精简指令集计算机的简称。

其指令集结构只有少数简单的指令, 使计算机硬件简化, 将CPU的时钟频率提高, 配合流水线结构可做到一个时钟周期执行一条指令, 使整个系性能超过CISC结构的计算机。

RISC指令系统的特点是:

- 选取使用频率最高的一些简单指令;
- 指令长度固定, 指令格式和寻址方式种类少;
- 只有取数据和存数据指令访问存储器, 其余指令的操作数在寄存器之间进行。